

問 4

以下は、アクセス記録テーブル（access_log）に対して、既存のreferer列の定義を「データ型VARCHAR(200)、NULL値を許可」で置き換えるためのSQL命令ですが、2点誤りがあります。誤りを指摘してください。

```
ALTER TABLE
  access_log
MODIFY WITH
  referer, VARCHAR(200), NULL
;
```

解答集

マスタ・プラクティス

Chapter
1基本的な検索テクニックを
理解しよう1-1 テーブルからすべての列を取り出したい
SELECT 命令

問 1

```
SELECT ①
*
FROM
  books ②
;
```

問 2

```
SELECT
* ①
FROM ②
  author
;
```

問 3

解答例は以下の通りです。

```
SELECT
*
FROM
  sales
;
```

問 4

以下の 2 点が誤りです。

- すべての列を表すのは「+」ではなく「*」です。
- SQL 命令の末尾は「:」ではなく「;」で表します。

なお、SQL 命令では大文字小文字は区別しません。本書では見易さへの考慮から予約語を大文字で表記していますが、問題のように小文字で記述しても構文上の誤りではありません。

以上を修正した SQL 命令は、以下の通りです。

```
select
*
from
  books
;
```

1-2 テーブルから特定の列を取り出したい
SELECT 命令

問 1

```
SELECT ①
  title
FROM
  books ②
;
```

問 2

```
SELECT
  p_name, ①
  price
FROM ②
  product
;
```

複数の列を指定する場合は、列名をカンマ (,) 区切りで記述します。

問 3

解答例は、以下の通りです。

```
SELECT
  l_name,
  f_name,
  class
FROM
  employee
;
```

問 4

以下の 2 点が誤りです。

- 「SELECTS」は「SELECT」です。
- 複数の列を指定する場合には、列名の間をカンマ (,) で区切る必要があります。

以上を修正した SQL 命令は、以下の通りです。

```
SELECT
  title,
  publish
FROM
  books
;
```

1-3 重複した行をあらかじめ取り除きたい
DISTINCT キーワード

問 1

```
SELECT DISTINCT ①
  prefecture
FROM
  quest ②
;
```

問 2

```
SELECT DISTINCT ①
  referer
FROM ②
  access_log
;
```

問 3

解答例は、以下の通りです。

```
SELECT DISTINCT
  l_name,
  f_name
FROM
  usr
;
```

複数の列を取り出す場合も、DISTINCT キーワードは常に SELECT 命令の直後に 1 回だけ指定します。

問 4

誤りは以下の 2 点です。

- 「ALL」は「DISTINCT」の誤りです。
- 取得列として、depart_id は不要です。depart_id を含めた場合、depart_id、class 列ごとに重複のないデータが取り出されることになります。

以上を修正した SQL 命令は、以下の通りです。

```
SELECT DISTINCT
  class
FROM
  employee;
```

1-4 条件に合致したレコードだけを
取り出したい WHERE 句

問 1

```
SELECT
  answer1, ①
  answer2
FROM
  quest
WHERE
  age >= 20 ②
;
```

問 2

```
SELECT
  isbn,
  title,
  publish
FROM
  books ①
WHERE
  publish IN ('日経 B P', '翔泳社')
  ②
;
```


この SQL 命令は、論理演算子 OR を使用して書き換えることも可能です。OR 演算子については、P.71 を参照してください。

問 3

```
SELECT
  l_name,
  f_name,
  email
FROM
  usr ①
WHERE
  prefecture <> '東京都' ②
;
```

問題文の条件を読み間違えないようにしてください。住所が東京都「でない」人を抽出するので、比較演算子は「=」ではなく「<>」を使用します。

問 4

解答例は、以下の通りです。

```
SELECT
  name,
  sex,
  prefecture
FROM
  quest
WHERE
  age BETWEEN 30 AND 39
;
```

この SQL 命令は、AND 演算子 (P.67) によっても書き換えが可能です。データベースによっては、BETWEEN 演算子よりも AND 演算子を使ったほうがパフォーマンスが良いものもあります。また、データベースによっては、BETWEEN 演算子は内部的に AND 演算子に置き換えられています。

問 5

解答例は、以下の通りです。

```
SELECT
  name,
  answer1,
  answer2
FROM
  quest
WHERE
  answered >= '2013-01-01'
;
```

問 6

解答例は、以下の通りです。

```
SELECT
  user_id,
  isbn
FROM
  rental
WHERE
  returned=0
;
```

P.36 のテーブル仕様を見れば、貸し出し中の定義は returned 列が 0 であることということが分かります。別解として「SELECT user_id, isbn FROM rental WHERE returned<>1;」でも間違いではありませんが、将来的にコードの意味が増えた場合に（たとえば、2 として「紛失」などという意味が追加された場合に）意図しない情報を取り出してしまふ危険もありますので、明確に「=0」で条件を限定する方が好ましいでしょう。

また、return 列にインデックス (P.247) が設定されている場合、<> 演算子を使用すると、インデックスが使われなくなってしまうので、パフォーマンスの観点からも = 演算子を利用すべきです。

問 7

解答例は、以下の通りです。

```
SELECT
  answer2
FROM
  quest
WHERE
  answer2 IS NOT NULL
;
```

この問 7 では、明確に「未定義」でないデータを、と指定していますので、解答例の通りで OK ですが、実際にデータを取り出す場合には、空文字列のデータが含まれている場合も除外するのが妥当でしょう。空文字列まで考慮した条件式の記述については、1-6 の問 3 の解答例 (P.271) をご覧ください。

問 8

以下の 2 点が誤りです。

- ・複数の列を指定する場合には、列名の間をカンマ (,) で区切る必要があります。
- ・「<=」は「<」です。

2 番目の誤りは文法的な誤りでないため、やや分かりにくかったかもしれません。問題文が「以下」ではなく「未満」と指示している点に注目してください。

以上を修正した SQL 命令は、以下の通りです。

```
SELECT
  title,
  publish,
  price
FROM
  books
WHERE
  price < 5000
;
```

1-5 あいまい条件に合致したレコードだけを取り出したい LIKE 演算子

問 1

```
SELECT
  * ①
FROM
  author
WHERE
  name LIKE '山田%' ②
;
```

前方一致パターンは「キーワード + %」で表現します。また、全列を取り出したい場合、列名の代わりにアスタリスク (*) を指定すればいいのでしたね。別解として、author テーブルに含まれる全列 (author_id, name, birth) を明示的に指定しても間違いではありません。

問 2

```
SELECT
  name
FROM ①
  quest
WHERE
  name NOT LIKE '%子' ②
;
```

後方一致パターンは「% + キーワード」で表現します。本問では、「子」で「終わらない」ものをという指定なので、LIKE 演算子ではなく、NOT LIKE 演算子を使用する点に注意してください。

問3

解答例は、以下の通りです。

```
SELECT
  isbn,
  title,
  price
FROM
  books
WHERE
  title LIKE 'SQL__';
```

この問3では、単純な前方一致検索ではなく、厳密に文字数が指定されている点に注意してください。任意の1文字を表すのはアンダースコア () です。この場合は2文字ですので、アンダースコアを2個続けて記述します。

問4

解答例は、以下の通りです。

```
SELECT
  l_name,
  f_name
FROM
  employee
WHERE
  l_name_kana LIKE 'ア%';
```

前方一致パターンは「キーワード + %」で記述します。

問5

誤りは以下の2点です。

- ・titleの後方のカンマ (,) が不要です。
- ・「_社」は「%社」の誤りです。

ワイルドカード「_」は「任意の1文字」を表わすものです。本問は単純な後方一致検索を表わしますので、「0文字以上の文字列」を表わす「%」を使用すべきです。

以上を修正したSQL命令は、以下の通りです。

```
SELECT
  title
FROM
  books
WHERE
  publish LIKE '%社';
```

1-6 複数条件に合致したレコードだけを
取り出したい 論理演算子

問1

```
SELECT
  *
FROM
  quest
WHERE
  sex='女' —①
AND
  age BETWEEN 20 AND 29 —②;
```

②は「(age>=20 AND age<30)」のように更に複数の条件式で記述しても構いませんが、範囲指定する場合には BETWEEN 演算子のほうがシンプルに記述できます。

問2

```
SELECT
  l_name,
  f_name,
  email
FROM
  usr
WHERE
  prefecture='東京都' —①
AND
  email LIKE '%@examples.com' —②;
```

「E-Mail アドレスのドメインが「examples.com」という条件が分かりにくく思われるかもしれませんが

が、要は「@examples.com」で終わるE-Mailアドレス」と条件内容を読みかえることができれば簡単です。LIKE 演算子で後方一致検索をしてみましょう。

問3

解答例は、以下の通りです。

```
SELECT
  answer2
FROM
  quest
WHERE
  answer2 IS NOT NULL
AND
  answer2 <> '';
```

「空でないもの」という条件がいささか曖昧ではありますが、要はなにかしら感想欄に記入があるものと考えれば、未定義 (NULL)、空文字列 () のいずれも取り除くのが順当でしょう。

Oracle では、空文字が NULL として処理されます。NULL は「=」「<>」演算子による比較はできませんので、「<>」という記述も不可です。

問4

解答例は、以下の通りです。

```
SELECT
  *
FROM
  employee
WHERE
  l_name='山田'
AND
  f_name='奈美';
```

問5

誤りは以下の2点です。

- ・OR 演算子と AND 演算子が逆転しています。
- ・「publish='秀和システム' OR publish='日経BP」

P」はカッコで括る必要があります。

したがって、正しいSQL命令は以下の通りです。3つ以上の条件式を連結する場合には、演算子の優先順位に注意するようにしてください。

```
SELECT
  isbn,
  title
FROM
  books
WHERE
  (
    publish='秀和システム'
    OR
    publish='日経BP'
  )
AND
  price < 3000;
```

1-7 特定の列を使って行を並べ替えたい
ORDER BY 句

問1

```
SELECT
  title,
  price
FROM
  books
WHERE
  price BETWEEN 2500 AND 3500 —①
ORDER BY
  price ASC —②;
```

①は <=、>=、AND 演算子を利用しても構いませんが、BETWEEN 演算子の方がよりシンプルに記述できます。ORDER BY 句でのソート順のデフォルトは ASC ですので、②でも ASC を省略しても構いませんが、コードの可読性を維持するという意味でも明示的に記述することをお勧めします。

問 2

```
SELECT
  l_name,
  f_name,
  prefecture
FROM
  usr
WHERE
  prefecture IN('東京都','千葉県',
               '神奈川県') ①
ORDER BY
  l_name_kana ASC, ②
  f_name_kana ASC
;
```

①は=、OR 演算子を利用しても構いませんが、IN 演算子の方がよりシンプルに記述できます。ORDER BY 句で複数のソートキーを指定する場合には条件同士をカンマ (,) 区切りで記述します。解答でも末尾のカンマを忘れないようにしてください。

問 3

解答例は以下の通りです。

```
SELECT
  *
FROM
  rental
WHERE
  returned=0
AND
  rental_date < '2012-12-01'
ORDER BY
  rental_date DESC
;
```

問題文ではやや曖昧に思われるかもしれませんが、2つの条件式は「論理積」で結ばれるべきものです。また、「新しい順 = 降順」の場合、DESC を指定します。

問 4

誤りは以下の2点です。

- ・「access_date IN ('2013-01-01','2013-01-31')」
→ 「access_date BETWEEN '2013-01-01' AND '2013-01-31」
- ・「referrer, ip_address DESC」 → 「referrer DESC, ip_address DESC」

複数のソートキーを持つ場合、それぞれのキーに対して ASC | DESC のいずれかを指定する必要があります。省略された場合には ASC とみなされますので、注意してください。

以上を修正した SQL 命令は、以下の通りです。

```
SELECT
  *
FROM
  access_log
WHERE
  access_date BETWEEN '2013-01-01'
  AND '2013-01-31'
ORDER BY
  referrer DESC, ip_address DESC
;
```

1-8 特定範囲のレコードのみを取り出したい LIMIT 句

問 1

```
SELECT
  title,
  publish_date
FROM
  books
ORDER BY ①
  publish_date DESC
LIMIT
  2, 5 ②
;
```

取得開始行は 0 スタートである点に注意します。LIMIT 句の構文は使用しているデータベースによって異なりますので、注意してください（詳細は P.84 の [Note] を参照）。

問 2

```
SELECT
  name,
  answer1,
  answer2
FROM
  quest
ORDER BY
  answered DESC ①
LIMIT
  0, 10
; ②
```

②が少々迷うかもしれませんが。問題文では先頭から取得するとあるので、取得開始行は省略しても良いのですが、もしも明記する場合には 0 を指定します。問 1 と同様、データベースごとの方言については、P.88 の [Note] を参照してください。

問 3

解答例は、以下の通りです。

```
SELECT
  user_id,
  isbn,
  rental_date
FROM
  rental
WHERE
  returned=0
ORDER BY
  rental_date ASC
LIMIT
  5
;
```

LIMIT 句は「LIMIT 0, 5」としても正解です。MySQL 以外のデータベースを利用している方は、P.84 の [Note] を参考に、それぞれの製品に対応した SQL も記述してみましょう。

問 4

誤りは以下の2点です。

- ・新しい順ですので、「ASC」は「DESC」です。
- ・「10,0」は、「10」または「0,10」です (MySQL の場合)。

LIMIT 句の構文は「取得開始行, 取得行数」ですので、注意してください。

以上を修正した SQL 命令は、以下の通りです。

```
SELECT
  page_id,
  referer,
  ip_address
FROM
  access_log
ORDER BY
  access_date DESC
LIMIT
  10
;
```

ただし、PostgreSQL では、LIMIT 句の構文として「取得行数, 取得開始行」が認められています (MySQL とはパラメータの順序が逆である点に注意)。したがって、問題の「LIMIT 10,0」は PostgreSQL 環境では正しいことになります。

1-9 特定の列を使ってデータをグルーピングしたい GROUP BY 句

問 1

```
SELECT
  sex, MAX(age), MIN(age) ①
FROM
  quest
GROUP BY ②
  sex
;
```

このように集計関数を複数列記述することも可能で

す。また、この問 1 では具体的な取得列を明示的に指示していませんが、グループ化する場合、「グループ化キーと集計列のみを取得列として指定できる」ことが分かっているれば、自ずと①の解答も限定できます。

問 2

```
SELECT
  prefecture, sex, AVG(answer1) —①
FROM
  quest
GROUP BY
  prefecture, sex —②
;
```

グループ化キーは複数指定することも可能です。その場合、グループ化キーをカンマ (,) 区切りで記述します。

問 3

解答例は、以下の通りです。

```
SELECT
  s_id,
  SUM(s_value)
FROM
  sales
GROUP BY
  s_id
;
```

問 4

解答例は、以下の通りです。

```
SELECT
  publish,
  MAX(price)
FROM
  books
GROUP BY
  publish
;
```

問 5

誤りは以下の 2 点です。

- ・「SUM(*)」は「COUNT(*)」です。
- ・「ORDER BY」は「GROUP BY」です。

COUNT 関数は該当するレコードの件数を取得するための集計関数です。COUNT(*) は、該当する行の数を取得します。COUNT(referer) のように、特定の列を指定しても良いのでは？と思われる方もいるかもしれませんが、この場合、referer 列が NULL でない行の数 (refere 列の値の数) が計算されますので、注意してください。

以上を修正した SQL 命令は、以下の通りです。

```
SELECT
  page_id,
  COUNT(*)
FROM
  access_log
GROUP BY
  page_id
;
```

1-10 列に別名を指定したい
AS キーワード

問 1

```
SELECT
  p_name AS 商品名, —①
  price AS 価格 —②
FROM
  product
ORDER BY
  price ASC
;
```

問 2

```
SELECT
  prefecture AS 都道府県名, —①
  AVG(age) AS 平均年齢
FROM
  quest
GROUP BY
  prefecture —②
;
```

グループ化については前項の復習です。グループ化した場合、アプリケーションなどからアクセスしやすいように、集計列に対しては原則として別名を付与すべきです。

問 3

解答例は、以下の通りです。

```
SELECT
  prefecture AS 都道府県名,
  COUNT(*) AS ユーザ数
FROM
  usr
GROUP BY
  prefecture
;
```

該当する行の数を求めるには、count(*) のようにします。列名としてワイルドカードであるアスタリスク (*) を指定する点に注目してください。

問 4

解答例は、以下の通りです。

```
SELECT
  publish AS 出版社,
  AVG(price) AS 価格平均
FROM
  books
GROUP BY
  publish
;
```

問 5

誤りは以下の 2 点です。

- ・ age 列はグループ化キーに含まれていませんので、取得列にも含めることができません。
- ・ 別名を指定するのは IS ではなく、AS キーワードの役割です。

このように、取得列の一部に対して別名を指定することも可能です。

以上を修正した SQL 命令は、以下の通りです。

```
SELECT
  prefecture,
  sex,
  AVG(answer1) AS 評価平均
FROM
  quest
GROUP BY
  prefecture,
  sex
;
```

1-11 文字列／日付データの
加工を行いたい文字列／日付関数

問 1

```
SELECT
  prefecture,
  ROUND(AVG(answer1)) AS 評価平均
FROM
  quest
GROUP BY
  prefecture —②
;
```

このように、関数の結果を引数にして異なる関数を呼び出すことも可能です。これを「関数の入れ子」と言います。

問 2

```
SELECT
  isbn AS ISBN コード,
  DATE_FORMAT(rental_date, ' %y/%m/%d') AS 貸出日 ①
FROM
  rental
WHERE
  returned=0
ORDER BY ②
  rental_date ASC
;
```

DATE_FORMAT 関数は MySQL 固有の関数です。そのほかデータベースでの構文については、P.109 の表を参考にしてください。

なお、この問 2 のように定型的な形式に整形したい場合、SQL Server では FORMAT 関数ではなく、CONVERT 関数で整形することもできます。

```
CONVERT(varchar, rental_date, 11) AS 貸出日
```

CONVERT 関数の第 3 引数にはフォーマットを表わすあらかじめ決められた番号を指定し、ここでの指定例では「yy/mm/dd」の形式に日付データを整形します。その他のフォーマット番号については、SQL Server のドキュメントを参照してください。

問 3

解答例は、以下の通りです。

```
SELECT
  CONCAT(l_name, f_name) AS 利用者名,
  CONCAT(prefecture, city,
    o_address) AS 住所
FROM
  usr
ORDER BY
  user_id DESC
;
```

解答例のように、CONCAT 関数は 3 つ以上の引数を持つことも可能です。そのほかデータベースでの構文については、P.109 の表を参考にしてください。

問 4

解答例は、以下の通りです。

```
SELECT
  publish,
  FLOOR(AVG(price))
FROM
  books
GROUP BY
  publish
;
```

複数の処理をする場合には、このように関数を入れ子で記述することも可能です。

SQLite では FLOOR 関数の代わりに、CAST 関数を使用して INTEGER 型に変換してください。

問 5

誤りは以下の 2 点です。

- ・「CONCAT(l_name, f_name)」は「CONCAT(l_name, f_name)」の誤りです。
- ・ORDER BY 句の「depart_id」は「depart_id DESC」の誤りです。

ORDER BY 句で複数のソートキーを指定する場合には、個々のキーに対して昇順／降順の別を指定する必要があります。省略した場合には ASC 指定とみなされます。

以上を修正した SQL 命令は、以下の通りです。

```
SELECT
  depart_id,
  class,
  CONCAT(l_name, f_name) AS 氏名
FROM
  employee
ORDER BY
  depart_id DESC,
  class DESC
;
```

そのほかデータベースでの構文については、P.109

の表を参考にしてください。

1-12 グループング結果に対して条件を付与したい HAVING 句

問 1

```
SELECT
  prefecture, AVG(age) ①
FROM
  quest
GROUP BY
  prefecture
HAVING
  AVG(age) BETWEEN 35 AND 49 ②
;
```

②は別解として「AVG(age)>=35 AND AVG(age)<50」でも可能です。HAVING 句でも WHERE 句と同様、論理演算子を用いた条件式を記述することができます。

問 2

```
SELECT
  prefecture, MAX(age) ①
FROM
  quest
WHERE ②
  sex='男'
GROUP BY
  prefecture
HAVING ③
  MAX(age)>60
;
```

一般的な絞り込み条件を指定するのは WHERE 句、集計列の絞り込みを行うのは HAVING 句の役割です。意外と混同しやすいところなので、改めて確認してみましょう。

問 3

解答例は、以下の通りです。

```
SELECT
  author_id,
  COUNT(isbn)
FROM
  author_books
GROUP BY
  author_id
HAVING
  COUNT(isbn) >= 3
;
```

グループごとのレコード数を求めるのは集計関数 COUNT の役割です。

問 4

解答例は、以下の通りです。

```
SELECT
  publish,
  category_id,
  COUNT(*)
FROM
  books
GROUP BY
  publish,
  category_id
HAVING
  COUNT(*) < 3
;
```

グループごとのレコード数を求めるのは集計関数 COUNT の役割です。

問 5

誤りは、以下の 2 点です。

- ・「ORDER BY」は「GROUP BY」の誤りです。
- ・2 番目の「WHERE」は「HAVING」の誤りです。

繰り返しになりますが、集計列の絞り込みを行うのは、HAVING 句の役割です。

以上を修正した SQL 命令は、以下の通りです。


```
SELECT
  depart_id,
  COUNT(*)
FROM
  employee
WHERE
  sex=2
GROUP BY
  depart_id
HAVING
  COUNT(*)>=3
;
```

問6

```
SELECT
  referer,
  count(*) —①
FROM
  access_log
WHERE
  access_date >= '2013-01-01' —②
GROUP BY
  referer
HAVING —③
  count(*) < 5
ORDER BY
  count(*) DESC
;
```

これまでに登場した一連のキーワードを活用した応用問題です。集計列について並べ替える場合にも、やはり ORDER BY 句を使用します。

1-13 条件によって値を変更したい CASE 演算子

問1

```
SELECT
  name AS 氏名,
  CASE
    answer1
    WHEN 3 THEN 'ためになった' —①
    WHEN 2 THEN '普通'
    WHEN 1 THEN '役に立たない'
    ELSE ''
  END AS 評価,
  answer2 AS 感想 —②
FROM
  quest
ORDER BY
  answered DESC —③
;
```

CASE 演算子の第2構文を利用した例です。このようなケースではあえて明示的に ELSE 句を指定する必要はありませんが、一応、想定外の値がセットされていた場合に備えて、空文字列を出力するようにしておきます。

問2

```
SELECT
  isbn,
  count(*) AS 貸出数, —①
  CASE
    WHEN count(*)>=10 THEN '好評' —③
    WHEN count(*)>= 5 THEN '普通' —③
    ELSE '不評'
  END AS 評価
FROM
  rental
GROUP BY —④
  isbn
;
```

CASE 演算子の第1構文を用いた例です。WHEN 句の条件式には集計列も含めることが可能です。

問3

解答例は、以下の通りです。

```
SELECT
  referer,
  count(*) AS カウント数,
  CASE
    WHEN count(*) >= 50 THEN 'A'
    WHEN count(*) >= 10 THEN 'B'
    ELSE 'C'
  END AS ランク
FROM
  access_log
GROUP BY
  referer
HAVING
  count(*)>=3
ORDER BY
  count(*) DESC
;
```

本文の復習ですが、HAVING、ORDER BY 句までを含めたやや複雑な SELECT 命令かもしれません。ただ、これまでの解説を念頭に順を追って記述していけば、1つひとつの句はそれほど難しいものではありません。間違えてしまった方は、過去の問いをもう一度見直してみましょう。

問4

解答例は以下の通りです。

```
SELECT
  author_id AS 著者ID,
  COUNT(isbn) AS カウント数,
  CASE
    WHEN count(*) >= 4 THEN '多い'
    WHEN count(*) >= 2 THEN '普通'
    ELSE '少ない'
  END AS 評価
FROM
  author_books
GROUP BY
  author_id
;
```

問5

誤りは以下の2点です。

- ・2つの「WHERE」は「WHEN」の誤りです。
- ・「ELSE THEN」は「ELSE」です。

CASE 演算子の構文はごく定型的なものです。他のキーワードと混同しやすい点もありますが、しっかりと覚えておいてください。

以上を修正した SQL 命令は、以下の通りです。

```
SELECT
  CONCAT(l_name, f_name),
  CASE
    WHEN class IN ('部長', '課長')
    THEN '管理職'
    WHEN class IN ('主任', '担当')
    THEN '総合職'
    ELSE '一般職'
  END AS 役職クラス
FROM
  employee
;
```


Chapter 2

より複雑な問い合わせを行おう

2-1 内部結合で複数テーブルのデータを結合したい INNER JOIN...ON 句

問 1

```
SELECT
  e.l_name, e.f_name, t.r_date,
  t.work_time ①
FROM
  employee AS e
  INNER JOIN ②
  time_card AS t
ON
  e.s_id=t.s_id ③
WHERE
  t.r_date BETWEEN '2012-12-01' AND
  '2012-12-31' ④
ORDER BY
  e.s_id ASC,
  t.r_date ASC
  ⑤
;
```

各列に対するテーブル名での修飾は（同名の列を除いては）任意ですが、本文でも紹介した通り、コードの読みやすさ、パフォーマンスの観点から、テーブル名は原則として明示することを強くお勧めします。

問 2

```
SELECT
  sh.s_name,
  sl.s_value
FROM
  sales AS sl ①
  INNER JOIN
  shop AS sh ②
ON
  sl.s_id = sh.s_id ③
WHERE
  sl.s_date='2012-12' ④
ORDER BY
  sl.s_value DESC ⑤
;
```

問 3

解答例は、以下の通りです。

```
SELECT
  sh.s_name,
  SUM(sl.s_value)
FROM
  sales AS sl
  INNER JOIN
  shop AS sh
ON
  sl.s_id = sh.s_id
WHERE
  sl.s_date BETWEEN '2012-01' AND
  '2012-12'
GROUP BY
  sh.s_id,
  sh.s_name
ORDER BY
  SUM(sl.s_value) ASC
;
```

2012 年の全売上高とありますので、単に GROUP BY 句でグループ化するだけではなく、WHERE 句

2-2 外部結合で複数テーブルのデータを結合したい

RIGHT JOIN...ON 句 / LEFT JOIN...ON 句

問 1

```
SELECT
  u.l_name,
  u.f_name,
  COUNT(r.id)
FROM
  rental AS r
  RIGHT JOIN ①
  usr AS u
ON
  r.user_id=u.user_id ②
GROUP BY
  u.user_id, u.l_name, u.f_name ③
ORDER BY
  COUNT(r.id) DESC ④
;
```

①は「RIGHT OUTER JOIN」でも可能です。GROUP BY 句には、本来のグルーピングキーである user_id 列だけではなく、取得列として含めなければならない l_name、f_name 列も必要である点に注意してください。

SQLite は、RIGHT JOIN 句に対応していません。P.142 を参考に、LEFT JOIN 句を使用してください。

問 2

```
SELECT
  d.depart_id,
  d.depart_name
FROM
  depart AS d
  LEFT JOIN
  employee AS e
ON
  d.depart_id = e.depart_id
WHERE
  e.depart_id IS NULL
;
```

やや特殊な問題だったかもしれません。左外部結合

で 2012 年 1 月から 12 月までのデータを取り出すのを忘れないでください。

また、P.95 でも述べたように、GROUP BY 句を利用した SELECT 命令では、グループ化キー、または集計列しか取得できない点に注意してください。

問 4

解答例は、以下の通りです。

```
SELECT
  u.l_name,
  u.f_name,
  COUNT(r.id)
FROM
  rental AS r
  INNER JOIN
  usr AS u
ON
  r.user_id=u.user_id
WHERE
  r.returned=0
GROUP BY
  r.user_id,
  u.l_name,
  u.f_name
HAVING
  COUNT(r.id)>0
ORDER BY
  COUNT(r.id) DESC
;
```


で2つのテーブルを結合した際に、右側のテーブルの外部キーがNULLであるかどうかを確認することで、対応する行が存在しないことを確認できます。

問3

解答例は、以下の通りです。

```
SELECT
  b.title,
  COUNT(r.isbn)
FROM
  books AS b
LEFT JOIN
  rental AS r
ON
  b.isbn = r.isbn
GROUP BY
  b.isbn,
  b.title
ORDER BY
  COUNT(r.isbn) DESC
;
```

問4

解答例は、以下の通りです。

```
SELECT
  e.l_name,
  e.f_name,
  AVG(t.work_time)
FROM
  employee AS e
LEFT JOIN
  time_card AS t
ON
  e.s_id = t.s_id
GROUP BY
  t.s_id,
  e.l_name,
  e.f_name
ORDER BY
  AVG(t.work_time) DESC
;
```

2-3 自己結合で同一テーブル内のデータを結合したい JOIN句+別名定義

問1

```
SELECT
  cp.c_id AS 前のページ,
  cn.c_id AS 現在のページ
FROM
  contents AS cp ①
INNER JOIN
  contents AS cn
ON
  cp.next_id = cn.c_id ②
ORDER BY
  cn.c_id ASC ③
;
```

2-3の本文の問いと似ていますが、今度は「次のページ」ではなく「前のページ」を取得するということで迷われるかもしれません。しかし、「次のページ」を「現在のページ」に、「現在のページ」を「前のページ」に置き換えるだけの話です。

問2

```
SELECT
  mc.title AS メニュー名,
  mn.title AS 親メニュー名
FROM
  menu AS mc ②
INNER JOIN
  menu AS mn
ON
  mc.parent = mn.page_id ④
ORDER BY
  mc.page_id ASC ⑤
;
```

典型的な自己結合の例です。自己結合を利用することで、このように再帰的、あるいは階層構造を持った情報をシンプルなコードで取り出すことができます。

問3

解答例は、以下の通りです。

```
SELECT
  CONCAT(e.l_name, e.f_name) AS
  社員氏名,
  CONCAT(eu.l_name, eu.f_name) AS
  上司氏名
FROM
  employee AS e
LEFT JOIN
  employee AS eu
ON
  e.b_id = eu.s_id
ORDER BY
  e.s_id ASC
;
```

問題文から上司を持たない（つまり、対応するeuテーブルもない）情報も取り出すということです。結合条件としてLEFT JOIN...ON句を指定します。また、取り出す列は、社員氏名と上司氏名という指定がありますので、CONCAT関数で列を連結しておく必要があります。データベースによっては、||演算子や+演算子を使用してください(P.109参照)。

問4

誤りは以下の2点です。

- ・「OUTER JOIN」は「INNER JOIN」の誤りです。
- ・「cp.next_id <> cn.c_id」は「cp.next_id = cn.c_id」の誤りです。

以上を修正した正しいSQL命令については、問1の解答を参照してください。

2-4 3つ以上のテーブルを結合したい (結合の応用) JOIN句の入れ子

問1

```
SELECT
  d.depart_name AS 所属部署名,
  CONCAT(e.l_name, e.f_name) AS 社員氏名,
  t.work_time AS 勤務時間 ①
FROM
  (
    employee AS e
    INNER JOIN ②
    depart AS d
    ON
    e.depart_id = d.depart_id ③
  )
  INNER JOIN ②
  time_card AS t
ON
  e.s_id = t.s_id ④
WHERE
  e.s_id = 'DA00001'
AND
  t.r_date BETWEEN '2012-12-01' AND
  '2012-12-31'
ORDER BY
  t.r_date ASC ⑤
;
```


問 2

```

SELECT
  cp.c_title AS 前コンテンツ名,
  cc.c_title AS 現コンテンツ名,
  cn.c_title AS 次コンテンツ名
FROM
  (
    contents AS cp
  INNER JOIN —②
    contents AS cc
  ON
    cp.next_id = cc.c_id —③
  )
INNER JOIN —②
  contents AS cn
ON
  cc.next_id = cn.c_id —④
ORDER BY
  cc.c_id ASC —⑤
;

```

自己結合を用いたテーブルの結合です。このようにすることで、たとえば、次の次の、または次の次のコンテンツを取得することも可能です。

問 3

解答例は、以下の通りです。

```

SELECT
  o.order_date AS 発注日,
  o.po_id AS 注文コード,
  CONCAT(u.l_name ,u.f_name) AS
  利用者氏名,
  p.p_name AS 商品名,
  p.price AS 商品単価,
  od.quantity AS 購入数
FROM
  (
    (
      order_main AS o
    INNER JOIN
      order_desc AS od
    ON
      o.po_id = od.po_id
    )
    INNER JOIN
      product AS p
    ON
      od.p_id = p.p_id
  )
INNER JOIN
  usr AS u
ON
  o.user_id = u.user_id
WHERE
  o.delivery_date IS NULL
ORDER BY
  o.order_date ASC,
  o.po_id ASC,
  od.p_id ASC
;

```

2-5 ある問い合わせ結果に基づいて、
検索したい サブクエリ

問 1

解答例は、以下の通りです。

```

SELECT
  answer1,
  answer2
FROM
  quest
WHERE
  age >
  (
    SELECT
      AVG(age)
    FROM
      quest
  )
ORDER BY
  answer1 ASC
;

```

サブクエリを利用した典型的な例です。サブクエリで、アンケート回答者の平均年齢を求めた上で、その結果を条件値として quest テーブルを検索します。

問 2

```

SELECT
  depart_id,
  depart_name
FROM
  depart —①
WHERE
  NOT EXISTS —②
  (
    SELECT
      *
    FROM
      employee
    WHERE
      employee.depart_id =
      depart.depart_id —③
  )
;

```

このように、主問い合わせとサブクエリとが相関関係にある問い合わせのことを相関サブクエリと言います。相関サブクエリでは、③のようにサブクエリから主問い合わせで利用しているテーブルを参照しているのが特徴です。

問 3

解答例は、以下の通りです。

```

SELECT
  sh.s_name,
  sl.s_value
FROM
  sales AS sl
INNER JOIN
  shop AS sh
ON
  sl.s_id=sh.s_id
WHERE
  sl.s_date='2012-12'
AND
  sl.s_value <
  (
    SELECT
      AVG(s_value)
    FROM
      sales
    WHERE
      s_date='2012-12'
  )
;

```

内部結合まで絡んでいるため、やや難易度が高くなっています。まずは、sales テーブルにのみ着目して目的の売上高を取得し、そのあとで店舗名を取得するようにすると、頭を整理しやすいでしょう。

問 4

解答例は、以下の通りです。

```
SELECT
  l_name,
  f_name
FROM
  usr
WHERE
  NOT EXISTS
  (
    SELECT
      *
    FROM
      rental
    WHERE
      rental.user_id=usr.user_id
  )
;
```

2-6 複数テーブルから取り出したデータを統合したい UNION 句

問 1

```
SELECT
  q.name_kana
FROM
  quest AS q
WHERE
  q.sex='女' ①
UNION ALL ②
SELECT
  CONCAT(e.l_name_kana,
         e.f_name_kana) ③
FROM
  employee AS e
WHERE
  e.sex=2
ORDER BY
  1 ASC ④
;
```

問題文に「重複を許可する」とありますので、UNION 演算子ではなく UNION ALL 演算子を使用します。また、集合演算子を用いる場合、結果両者の列数は同一でなければなりませんので、employee テーブルの氏・名は CONCAT 関数（または || 演算子や+演算子）で文字列連結しておく必要があります。

問 2

```
SELECT
  u.l_name || u.f_name ①
FROM
  usr AS u
EXCEPT ②
SELECT
  q.name
FROM
  quest AS q ③
;
```

このように片方の結果に含まれていない情報を取り出すには、差集合を表わす EXCEPT 演算子を使用

します。EXCEPT 演算子は、MySQL では使用できませんので注意してください。Oracle では、同等の機能として MINUS 演算子を提供しています。

問 3

解答例は、以下の通りです。

```
SELECT
  CONCAT(u.l_name_kana,
         u.f_name_kana)
FROM
  usr AS u
WHERE
  u.prefecture='千葉県'
UNION
SELECT
  q.name_kana
FROM
  quest AS q
WHERE
  q.prefecture='千葉県'
ORDER BY
  1 DESC
;
```

問 4

誤りは以下の 2 点です。

- ・重複を含まないデータなので、「UNION ALL」は「UNION」です。
- ・「u.l_name_kana, u.f_name_kana」は「CONCAT(u.l_name_kana, u.f_name_kana)」です。

UNION 演算子などの集合演算子を利用する場合には、結果双方の列数、データ型が等しくなければなりません。

以上を修正した SQL 命令は、以下の通りです。

```
SELECT
  a.name_kana
FROM
  author AS a
UNION
SELECT
  CONCAT(u.l_name_kana,
         u.f_name_kana)
FROM
  usr AS u
ORDER BY
  1 DESC
;
```

問 5

解答例は、以下の通りです。

```
SELECT
  l_name || f_name
FROM
  usr AS u
INTERSECT
SELECT
  q.name
FROM
  quest AS q
;
```

「双方に含まれるもの」を抽出するには、積集合 (INTERSECT) を使用します。INTERSECT 演算子は、MySQL では使用できませんので、注意してください。

Chapter 3 データの登録／更新／削除を行おう

3-1 新規のデータを挿入したい INSERT 命令

問 1

```
INSERT INTO ①
books
VALUES ②
(
  '4-8833-0000-4',
  'SQL ミニ ',
  1000,
  '日経BP ',
  '2011-03-22 ',
  'S2222'
);
```

問 2

```
INSERT INTO
author ①
VALUES
(
  'I0001',
  '井田健二',
  'イダケンジ',
  NULL
);
```

問 3

解答例は以下の通りです。

```
INSERT INTO
depart
VALUES
(
  'E03',
  '第三営業部'
);
```

問 4

以下の 2 点が誤りです。

- ・「INSERT」は「INSERT INTO」の誤りです。
- ・VALUES 句で指定する値リストで、文字列はシングルクォート (') で括る必要があります。

以上を修正した SQL 命令は、以下の通りです。

```
INSERT INTO
product
VALUES
(
  'SB000000001',
  '黒スタンプ',
  1250
);
```

3-2 列指定で新規のデータを挿入したい INSERT 命令

問 1

```
INSERT INTO
quest
(
  name,
  name_kana,
  sex,
  prefecture,
  answer1,
  answer2,
  answered
)
VALUES
(
  '佐々木三郎',
  'ササキサブロウ',
  '男',
  '東京都',
  3,
  '興味深い内容です。',
  NOW()
);
```

Oracle、SQL Server では NOW 関数の代わりに、CURRENT_TIMESTAMP 関数を使用します (P.109 も参照)。いずれも後ろのカッコは不要ですので、注意してください (カッコをつけた場合にはエラーとなります)。SQLite では DATETIME 関数を使用します。

なお、Oracle には自動連番 (AUTO_INCREMENT) という概念がないため、シーケンスを利用します。よって、Oracle では以下のように id 列に対して、明示的にシーケンスの次の値をセットする必要があります。シーケンスの値は、「シーケンス名.NEXTVAL」で取得できます (PostgreSQL も AUTO_INCREMENT という概念はありませんが、代わりに、内部的にシーケンスを利用して連番を管理する SERIAL という擬似型を持っていますので、シーケンスを意識する必要はありません)。

```
INSERT INTO
quest
(
  id,
  name,
  name_kana,
  sex,
  prefecture,
  answer1,
  answer2,
  answered
)
VALUES
(
  quest_seq.NEXTVAL,
  '佐々木三郎',
  'ササキサブロウ',
  '男',
  '東京都',
  3,
  '興味深い内容です。',
  CURRENT_TIMESTAMP
);
```

quest_seq は、あらかじめ用意されたシーケンスの名前であるものとします。

問 2

```
INSERT INTO ①
rental
(
  user_id,
  isbn,
  rental_date
)
VALUES ②
(
  'B200502',
  '4-0010-0000-0',
  CURRENT_DATE
);
```

今日の日付を求めるには、MySQL、PostgreSQL、Oracle では CURRENT_DATE 関数 (後ろのカッ

コは不要)を、SQLite では DATE 関数を、SQL Server では CURRENT_TIMESTAMP 関数をそれぞれ使用してください。自動連番に関する注意は、問 1 と同様です。

問 3

解答例は、以下の通りです。

```
INSERT INTO
  author
(
  author_id,
  name,
  name_kana,
  birth
)
VALUES
(
  'U0001',
  '内田幸子',
  'ウチダサチコ',
  '1973-04-10'
);
```

前項で学んだ構文にしたがって、列名を省略して記述することも可能です。復習がてらに両方の記述を試してみましょう。

問 4

誤りは以下の 2 点です。

- ・「(s_id l_name depart_id last_update)」は「(s_id, l_name, depart_id, last_update)」の誤りです。
- ・「VALUE」は「VALUES」の誤りです。

列名は、カンマ (,) 区切りで指定する必要があります。

以上を修正した SQL 命令は、以下の通りです。

```
INSERT INTO
  employee
(
  s_id,
  l_name,
  depart_id,
  last_update
)
VALUES
(
  'WA00001',
  '和田',
  'J01',
  CURRENT_DATE
);
```

CURRENT_DATE 関数に関する注意は、問 2 と同様です。

3-3 検索結果を他のテーブルに挿入したい
INSERT 命令/サブクエリ

問 1

```
INSERT INTO
  time_card
(
  s_id,
  r_date,
  work_time
)
SELECT ①
  s_id,
  CURRENT_DATE,
  0
FROM
  employee
WHERE
  class IN
  (
    '主任',
    '担当',
    'アシスタント' ②
  )
;
```

INSERT 命令における副問い合わせは、VALUES 句の代わりに使用します。
CURRENT_DATE に関する注意は、3-2 の問 2 と同様です。

問 2

解答例は、以下の通りです。

```
INSERT INTO
  sales
(
  s_id,
  s_date,
  s_value
)
SELECT
  s_id,
  DATE_FORMAT(CURRENT_DATE, '%Y-%m'),
  0
FROM
  shop
;
```

DATE_FORMAT 関数は、MySQL で用意された日付関数です。他のデータベース製品では使用できなかったり、異なる名前の関数が用意されていたりしますので、P.109 を参考に適宜書き換えてください。

3-4 既存データを更新したい
UPDATE 命令

問 1

```
UPDATE ①
  books
SET
  isbn=CONCAT('ISBN', isbn), ②
  price=price/1.05 ③
;
```

UPDATE 命令において SET 句の列の順番に特に決まりはありません。したがって、②と③とは逆でも正解です。

CONCAT 関数は MySQL 固有の関数です。使用しているデータベースによって異なりますので、P.109 の表に従って、適宜書き換えてください。

問 2

```
UPDATE
  usr ①
SET
  email=NULL ②
;
```

NULL はそれ自体意味を持つキーワードですので、「NULL」のようにシングルクォートで括弧「ない」ように注意してください。

問 3

解答例は、以下の通りです。

```
UPDATE
  product
SET
  price=price*1.1
;
```

問 4

解答例は、以下の通りです。

```
UPDATE
  books
SET
  isbn=REPLACE(isbn, 'ISBN', '')
;
```

もしも PostgreSQL を使用している場合には、REPLACE 関数の代わりに TRANSLATE 関数を使用してください。

問 5

誤りは以下の 2 点です。

- ・「UPDATE INTO」は「UPDATE」の誤りです。
- ・「VALUES」は「SET」の誤りです。

以上を修正した SQL 命令は、以下の通りです。

```
UPDATE
  quest
SET
  sex=REPLACE(sex, '性', '')
;
```

問 6

```
UPDATE
  employee
SET
  class=
  (
    CASE ①
      class ②
      WHEN '部長' THEN 'リーダ'
      WHEN '課長' THEN 'マネージャ'
      ELSE class ②
    END
  )
;
```

P.121 でも利用した CASE 演算子を用いて、簡単な条件分岐を含めた更新ができます。

Oracle では、WHEN 句で文字列リテラルを利用する場合、「N 部長」のように N 接頭辞を付けて指定してください。さもないと、「SQL エラー:ORA-12704: キャラクタ・セットが一致しません。」のようなエラーが表示されます。

3-5 特定の条件に合致するデータを更新したい UPDATE 命令

問 1

```
UPDATE
  books ①
SET
  publish='WINGS 出版' ②
WHERE
  publish='山田出版' ③
;
```

問 2

```
UPDATE
  employee
SET
  class='主任', ①
  last_update=CURRENT_DATE ②
WHERE
  l_name='山田' AND f_name='奈美' ③
;
```

CURRENT_DATE 関数については、使用しているデータベースによって異なりますので、P.109 の表に従って、適宜書き換えてください。

問 3

解答例は以下の通りです。

```
UPDATE
  books
SET
  price=price*0.9
WHERE
  publish='日経BP'
;
```

問 4

解答例は以下の通りです。

```
UPDATE
  rental
SET
  returned=9
WHERE
  rental_date < '2012-03-31'
AND
  returned=0
;
```

問 5

誤りは以下の 2 点です。

- ・「rental_date CURRENT_DATE」は「rental_date=CURRENT_DATE」の誤りです。
- ・「OR」は「AND」の誤りです。

CURRENT_DATE 関数は必ずしもすべてのデータベースで利用できるとは限りません。データベースごとの関数については、P.109 を参照してください。以上を修正した SQL 命令は、以下の通りです。

```
UPDATE
  rental
SET
  rental_date=CURRENT_DATE
WHERE
  user_id='B200405'
AND
  isbn='4-0010-0000-1'
;
```

3-6 ある問い合わせ結果に基づいて、更新したい

UPDATE 命令/サブクエリ

問 1

```
UPDATE
  employee
SET
  depart_id='Z99' ①
WHERE
  depart_id
NOT IN ②
(
  SELECT
    depart_id ③
  FROM
    depart
)
;
```

問 2

```
UPDATE ①
  employee
SET
  depart_id='Z99'
WHERE
  NOT EXISTS ②
(
  SELECT
    *
  FROM
    depart
  WHERE
    depart.depart_id=employee.depart_id ③
)
;
```


問 3

```

UPDATE
  books
SET
  title —①
=
  CONCAT(
    title, —②
    (
      SELECT
        category_name —③
      FROM
        category
      WHERE
        category.category_id=
        books.category_id —④
    )
  )
;

```

相関サブクエリを用いた更新命令です。このように、相関サブクエリを用いることで、外部テーブルの値を用いて、現在のテーブルの値を更新することができます。

CONCAT 関数は MySQL 固有の関数です。使用しているデータベースによって異なりますので、P.109 の表に従って、適宜書き換えてください。

問 4

解答例は、以下の通りです。

```

UPDATE
  books
SET
  category_id='XXXXX'
WHERE
  isbn
IN
(
  SELECT
    isbn
  FROM
    rental
  WHERE
    returned=9
)
;

```

3-7 既存データを削除したい
DELETE 命令

問 1

```

DELETE FROM —①
  rental
WHERE
  rental_date<='2011-12-31' —②
AND
  returned=1 —②
;

```

問 2

```

DELETE FROM
  access_log —①
WHERE
  access_date<'2012-06-01' —②
OR
  (referer IS NULL OR referer='') —②
;

```

リンク元の URL が空という条件はやや曖昧ですが、P.271 でも紹介したとおり、NULL、空文字の双

3-8 ある問い合わせ結果に基づいて、
削除したい

DELETE 命令／サブクエリ

問 1

```

DELETE FROM —①
  employee
WHERE
  depart_id
NOT IN —②
(
  SELECT
    depart_id
  FROM
    depart —③
)
;

```

問 2

```

DELETE FROM
  employee
WHERE
  NOT EXISTS —①
(
  SELECT
    depart.depart_id
  FROM
    depart —②
  WHERE
    depart.depart_id=employee.depart_id —③
)
;

```

方を含めるのが妥当でしょう。条件式を括弧カッコはなくても意味は同じですが、意味ある条件式同士は明確にカッコで括弧することで、視覚的にも分かりやすくなります。

問 3

解答例は、以下の通りです。

```

DELETE FROM
  employee
WHERE
  retired=1
AND
  last_update<='2007-03-31'
;

```

問 4

誤りは以下の 2 点です。

- ・「DELETE」は「DELETE FROM」の誤りです。
- ・「AND」は「OR」の誤りです。

以上を修正した SQL 命令は、以下の通りです。

```

DELETE FROM
  books
WHERE
  category_id='Z9999'
OR
  category_id IS NULL
;

```


問 3

解答例は、以下の通りです。

```
DELETE FROM
  sales
WHERE
  s_id
NOT IN
(
  SELECT
    s_id
  FROM
    shop
)
```

相関サブクエリを用いた場合、以下のように書き換えることも可能です。

```
DELETE FROM
  sales
WHERE
  NOT EXISTS
(
  SELECT
    shop.s_id
  FROM
    shop
  WHERE
    shop.s_id=sales.s_id
)
```

問 4

解答例は、以下の通りです。

```
DELETE FROM
  books
WHERE
  isbn
IN
(
  SELECT
    isbn
  FROM
    rental
  WHERE
    returned=9
)
```

Chapter

4

データベース構造を操作しよう

4-1 テーブルを作成したい
CREATE TABLE 命令

問 1

```
CREATE TABLE
  author —①
(
  author_id CHAR(5) PRIMARY KEY, —②
  name VARCHAR(30),
  name_kana VARCHAR(100),
  birth DATE —③
)
```

主キー (PRIMARY KEY) は、列定義の末尾のオプションとして指定するほか、列フラグとして指定することも可能です。主キーが複合キー (複数列から構成されるキー) でない場合には、列フラグとして定義してしまった方がシンプルです。

問 2

```
CREATE TABLE —①
  order_desc
(
  po_id INT NOT NULL,
  p_id CHAR(10) NOT NULL,
  quantity INT, —②
  PRIMARY KEY (po_id, p_id) —③
)
```

問 1 と異なり、主キーが複合キーとなっていますので、オプションとして定義しています。

問 3

解答例は以下の通りです。

```
CREATE TABLE
  sales
(
  s_id CHAR(7) NOT NULL,
  s_date DATE NOT NULL,
  s_value INT,
  PRIMARY KEY (s_id, s_date)
)
```

問 4

解答例は、以下の通りです。

```
CREATE TABLE
  books
(
  isbn CHAR(17),
  title VARCHAR(255),
  price INT,
  publish VARCHAR(30),
  publish_date DATE,
  category_id CHAR(5),
  PRIMARY KEY (isbn)
)
```

問 5

誤りは以下の 2 点です。

- ・「CREATE TABLE INTO」は「CREATE TABLE」の誤りです。
- ・「AUTO_INCREMENT, PRIMARY KEY,」は「AUTO_INCREMENT PRIMARY KEY,」の誤りです。

列フラグを複数指定する場合には空白区切りで指定

します。

以上を修正した SQL 命令は、以下の通りです。

```
CREATE TABLE
rental
(
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id CHAR(7),
  isbn CHAR(13),
  rental_date DATE,
  returned SMALLINT DEFAULT 0
);
```

Oracle では AUTO_INCREMENT は使用できませんので、代わりにシーケンスを別に用意する必要があります。

4-2 新規にインデックスを作成したい CREATE INDEX 命令

問 1

```
CREATE INDEX —①
ind_rental
ON —②
rental
(
  rental_date —③
);
```

問 2

```
CREATE INDEX —①
ind_order
ON
order_main —②
(
  order_date, delivery_date —③
);
```

複合インデックスを作成する場合は、構成する列名をカンマ区切りで記述します。

問 3

解答例は、以下の通りです。

```
CREATE INDEX
ind_product
ON
product
(
  p_name
);
```

問 4

誤りは以下の 2 点です。

- ・「IN」は「ON」の誤りです。
- ・列名の区切りはカンマで記述する必要があります。

以上を修正した SQL 命令は、以下の通りです。

```
CREATE INDEX
ind_usr
ON
usr
(
  prefecture,
  city,
  o_address
);
```

4-3 既存テーブルに列や制約条件を追加したい ALTER TABLE 命令

問 1

```
ALTER TABLE —①
books
ADD
  pages INT DEFAULT 0 NOT NULL, —②
ADD
  rating CHAR(1) DEFAULT 'B' NOT
  NULL
AFTER
  category_id —③
;
```

ALTER TABLE 命令で複数の列を挿入する場合には、ADD 句を列記します。ただし、SQL Server では 2 つ目の ADD は不要で、そのまま列定義を続けて記述します。SQLite では、一度に複数列の追加はできません。

列の追加先は「テーブル定義の一番最後」とありますので、P.36 のフィールド・レイアウトから category_id 列の後方であると判断します。ただし、MySQL 以外のデータベースでは、AFTER 句による挿入箇所の設定はできませんので、注意してください。

問 2

```
CREATE TABLE —①
sales
(
  s_id CHAR(5) NOT NULL, —②
  s_date DATE NOT NULL,
  s_value INT DEFAULT 0
);
ALTER TABLE —③
sales
ADD
  PRIMARY KEY (s_id, s_date) —④
;
```

前半は前項の復習です。ALTER TABLE 命令を利用することで、列だけでなく、制約条件も追加でき

ます。ただし、SQLite は未対応です。

問 3

解答例は、以下の通りです。

```
ALTER TABLE
books
ADD
  sales INT
AFTER
  category_id
;
```

AFTER 句は MySQL 固有の構文です。他のデータベースでは利用できません。

問 4

解答例は、以下の通りです。

```
ALTER TABLE
usr
ADD
  sex VARCHAR(5) DEFAULT '男',
AFTER
  f_name_kana,
ADD
  job VARCHAR(30)
AFTER
  sex
;
```

AFTER 句は MySQL 固有の構文です。他のデータベースでは利用できません。

問 5

誤りは以下の 2 点です。

- ・「ADD TO」は「ADD」の誤りです。
- ・「BEFORE」は「AFTER」の誤りです。

以上を修正した SQL 命令は、以下の通りです。


```
ALTER TABLE
  employee
ADD
  email VARCHAR(255)
AFTER
  depart_id
;
```

4-4 既存テーブル上の列や制約条件を変更したい ALTER TABLE 命令

問 1

```
ALTER TABLE ①
  books
MODIFY
  publish VARCHAR(100) NULL ②
;
```

ただし、SQLite は未対応です。また、SQL Server、PostgreSQL では、MODIFY 句の代わりに ALTER COLUMN 句を使用します。

問 2

```
ALTER TABLE
  employee ①
MODIFY ②
  l_name VARCHAR(50) NOT NULL,
MODIFY ③
  f_name VARCHAR(50) NOT NULL
;
```

ただし、SQLite は未対応です。また、SQL Server、PostgreSQL では、MODIFY 句の代わりに ALTER COLUMN 句を使用します。

MODIFY 句で複数の列定義をまとめて変更する場合には、MODIFY 句をカンマ区切りで列記してください。ただし、SQL Server では列定義の変更をまとめて複数列に対して行うことはできませんので、1 列ずつ実行する必要があります。

問 3

解答例は、以下の通りです。

```
ALTER TABLE
  author
MODIFY
  name VARCHAR(100) DEFAULT ''
;
```

ただし、SQLite は未対応です。また、SQL Server、PostgreSQL では、MODIFY 句の代わりに ALTER COLUMN 句を使用します（ただし、デフォルト値の指定は不可）。

問 4

誤りは以下の 2 点です。

- ・「MODIFY WITH」は「MODIFY」の誤りです。
- ・「referer,VARCHAR(200), NULL」は「referer VARCHAR(200) NULL」の誤りです。

列定義は「列名 データ型 列フラグ」のように空白区切りで記述します。

以上を修正した SQL 命令は、以下の通りです。

```
ALTER TABLE
  access_log
MODIFY
  referer VARCHAR(200) NULL
;
```


索引

記号

% 64, 103
 * 103
 / 103
 ^ 109
 _ 65
 || 109
 + 103, 109
 - 103
 < 57
 <= 57
 <> 57
 = 57
 > 57
 >= 57

A

ABS 109
 Access 7
 ADD 255
 AFTER 255
 ALL 52
 ALTER COLUMN 261
 ALTER TABLE 255, 261
 AND 71
 AS 100, 101, 132
 ASC 80, 109
 ASCII 109

AUTO_INCREMENT 239
 AVG 95

B

BEGIN 229
 BETWEEN 57, 268

C

CASE 122
 CAST 109
 CEIL 109
 CEILING 109
 CHAR 109
 CHAR_LENGTH 109
 Character User Interface 24
 CHARINDEX 109
 CHR 109
 COMMIT 229
 CONCAT 109
 COUNT 95
 CREATE INDEX 247
 CREATE TABLE 239
 CREATE TABLE ...AS 200
 CREATE VIEW 170
 CROSS JOIN 151
 CUI 24
 CURRENT_DATE 109
 CURRENT_TIME 109

D

Data Control Language 4
 Data Definition Language 4
 Data Manipulation Language 4
 DataBase 2
 DATE 109
 DATE_ADD 109
 DATE_FORMAT 109
 DB 2
 DB2 7
 DCL 4
 DDL 4
 DEFAULT 239
 DELETE 223, 232
 DESC 80
 DISTINCT 52, 267
 DML 4
 DROP 255
 DROP COLUMN 256
 DROP DATABASE 230
 DROP INDEX 248
 DROP TABLE 241

E

EXCEPT 178
 EXISTS 166, 218, 232

F

FLOOR 109
 FOREIGN KEY 11, 241

FROM 44
 FULL OUTER JOIN 151

G

GROUP BY 94, 146

H

HAVING 115

I

IN 57
 INDEX 241, 247
 INNER JOIN...ON 132
 INSERT 187, 192, 199
 INSTR 109
 INT 109
 INTERSECT 177
 IS NULL 57

K

Key-Value 5

L

LEFT JOIN...ON 141
 LEFT OUTER JOIN 141
 LEN 109
 LENGTH 109
 LIKE 57, 64

LIMIT 87
 LOWER 109
 LTRIM 109

M

MAX 95
 MIN 95
 MINUS 178
 MODIFY 261
 MySQL 7, 10, 18
 MySQL Monitor 24

N

NOT EXISTS 167
 NOW 109
 NULL 57, 81, 187, 239

O

ODBC 7
 ON 134, 250
 Open DataBase Connectivity 7
 OR 71
 Oracle 7
 ORDER BY 79, 88

P

PATH 22
 phpMyAdmin 29
 POSITION 109

PostgreSQL 7
 POW 109
 POWER 109
 PRIMARY KEY 10, 239, 241

R

RAND 109
 RDB 2
 REFERENCES 239
 Relational DataBase 2
 RENAME AS 261
 RENAME TO 261
 REPLACE 109
 RIGHT JOIN...ON 141
 RIGHT OUTER JOIN 141
 ROLLBACK 229
 ROUND 109
 rownum 88
 RTRIM 109

S

SELECT 43, 47
 SELECT INTO 200
 SERIAL 241
 SET 206
 SHOW 25
 SHOW FIELDS 26
 SHOW TABLES 26
 SQL 2, 4
 SQL 2003 5
 SQL CLR 161

SQL Server 7
 SQL92 4
 SQL99 4
 SQLite 7
 SQR 109
 SQRT 109
 Stored Procedure 161
 SUBSTR 109
 SUBSTRING 109
 SUM 95

T

TABLE 230
 TIME 109
 TOP 88
 Transact SQL 161
 TRANSLATE 109
 TRUNCATE 230

U

UNION 175
 UNION ALL 177
 UNIQUE 239, 241, 247
 UNIQUE 制約 12
 UPDATE 200, 206, 211
 UPPER 109
 USE 25
 USING 143

V

VALUES 187, 192

W

WHERE 56, 132, 211

あ行

アスタリスク 34, 43
 インデックス 241, 247
 エイリアス 101
 演算子 56
 オブジェクト指向データベース 5
 オプション 239

か行

カード型データベース 5
 外部キー 11
 外部結合 132, 136, 141, 142
 仮想テーブル 170
 カラム 8
 環境変数 22
 関数 94
 関数従属 14
 関数の入れ子 275
 グループ化 94
 交差結合 151
 降順 80
 後方一致検索 65

コミット 228

さ行

索引 247
 差集合 178
 サブクエリ 88, 165, 200, 217, 232
 算術演算子 57, 102
 算術関数 108, 109
 参照制約 11
 サンプルデータベース 18, 33
 シーケンス 8, 241
 自己結合 151
 集計関数 94, 95
 主キー 10, 239
 昇順 80
 冗長化 17
 数値型 10
 ストアドプロシージャ 8
 正規化 13
 制約条件 9
 積集合 178
 前方一致検索 65
 相関サブクエリ 167, 218, 232
 ソート 79

た行

第 1 正規化 13
 第 2 正規化 14
 第 3 正規化 15
 第 4 正規化 16
 第 5 正規化 16

単項演算子 58
 ツリー型データベース 5
 データベース 2
 データベース管理システム 2
 データベースサーバ 5
 データ型 9
 データ制御言語 4
 データ操作言語 4
 データ定義言語 4
 テーブル 4, 6, 8
 等価結合 132
 トランザクション 228
 トリガー 8

な行

内部結合 132
 並べ替え 79
 ネットワーク型データベース 5

は行

バイナリ型 10
 非 NULL 制約 12
 比較演算子 57
 非正規化 13, 17
 左外部結合 141, 151
 日付型 9, 10
 日付関数 108, 109
 ビュー 8, 88, 171
 フィールド 8
 複合インデックス 250
 複合キー 239

副問い合わせ 165
 ブラックボックス 3
 ボイスコード (BC) 正規化 16

ま行

右外部結合 141, 151
 文字列型 10
 文字列関数 108, 109

や行

ユニークキー 239

ら行

リレーショナルデータベース 2, 5, 6
 リレーションシップ 5, 11
 論理演算子 57, 71
 論理積 72
 論理和 72
 レコード 8
 列フラグ 239
 ロールバック 228

わ行

ワイルドカード 64

■ 著者紹介 ■

山田 祥寛 (やまだ よしひろ)

静岡県榛原町生まれ。一橋大学経済学部卒業後、NECにてシステム企画業務に携わるが、2003年4月に念願かなってフリーライターに転身。Microsoft MVP for ASP/ASP.NET。執筆コミュニティ「WINGSプロジェクト」の代表でもある。

主な著書に、「アプリを作ろう! HTML5入門~HTML5+CSS3+JavaScriptで学ぶAndroid/iPhoneアプリ作成」(日経BP社)、「10日でおぼえる入門教室シリーズ(jQuery・SQL Server・ASP.NET・JSP/サーブレット・PHP・XML)」 「独習シリーズ(サーバサイドJava・PHP・ASP.NET)」 「PEAR入門」 「Smarty入門」 (以上、翔泳社)、「はじめてのAndroidアプリ開発 Android OS4対応版」(秀和システム)、「JavaScript本格入門」 「Ruby on Rails 3 アプリケーションプログラミング」 (以上、技術評論社) など。また、@IT(.NET、Windows)、CodeZineなどのサイトにて連載などでも記事を執筆中。最近では、IT関連技術の取材、講演まで広く手がける毎日である。最近の活動内容は、著者サイト (<http://www.wings.msn.to/>) にて。

■ カバー・本文デザイン 石田 昌治 (株式会社マップス)

■ DTP 株式会社マップス

本書はソシムより刊行されていた「書き込み式SQLのドリルドンドン身に付く、スラスラ書ける」の内容を一新、再構成したものです。

書き込み式 SQLのドリル 改訂新版

2013年4月22日 初版発行

2015年8月6日 初版2刷発行

著者 山田 祥寛
発行者 村上 広樹
発行 日経BP社
東京港区白金1-17-3 〒108-8646
発売 日経BPマーケティング
東京港区白金1-17-3 〒108-8646
印刷・製本 図書印刷株式会社

ISBN978-4-8222-9621-6

© 2013 YAMADA, Yoshihiro
Printed In Japan

● 本書の一部または全部について、個人で使用するほかは、著作権上、著者および日経BP社の承諾を得ずに無断で複写/複製することは禁じられております。購入者以外の第三者による電子データ化および電子書籍化は、私的使用を含め一切認められておりません。

● 本書の内容の運営によっていかなる障害が生じても、著者および日経BP社のいづれも一切の責任を負いかねますので、あらかじめご了承ください。

● 本書に関するサポート情報は、下記Webページをご参照ください。なお、本書の範囲を超えるご質問にはお答えできませんので、あらかじめご了承ください。

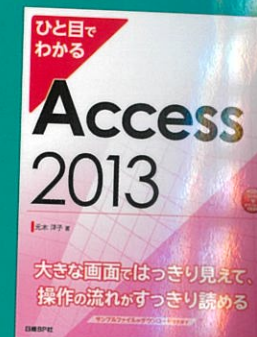
<http://www.wings.msn.to/>

日経BP社のデータベース関連書籍



ひと目でわかる SQL Server 2012

エディフィスラーニング株式会社 沖 要知 著
定価(本体3400円+税)
ISBN978-4-8222-9476-2



ひと目でわかる Access 2013

元木 洋子 著
定価(本体1600円+税)
ISBN978-4-8222-9489-2

カバーデザイン ● 石田昌治(株)マップス